ALUMNI CONNECT

PROJECT DOCUMENTATION

A platform helping students and alumni connect based on shared interests, skills, and organizations for networking, mentoring, and placements.

Project Report submitted in partial fulfilment of the requirements for the course

DATA STRUCTURES AND ALGORITHMS

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Submitted by

Team Members: RISHI A (2520030268)

SRI CHARAN (2520030093)

SATYA CHARAN (2520030368)

Team Number: [19] Section: [8]

Under the Guidance of

Dr. S. Vinay Kumar

Associate Professor

Department of Computer Science and Engineering

Academic Year: 2026–2027

DECLARATION

We declare that the project report entitled “Alumni Connect” is a record of work prepared by our team for academic purposes. The report describes the proposed design and planned functionality of a platform that helps students and alumni discover relevant connections for mentoring, networking, and career opportunities. Any implementation details and results should be updated to reflect the features actually completed by the team.

Team Members: RISHI A (2520030268)

SRI CHARAN (2520030093)

SATYA CHARAN (2520030368)

Signatures: _____

CERTIFICATE

This is to certify that the project report entitled “Alumni Connect” has been prepared by the students listed below under the guidance of [Guide name], in partial fulfilment of the requirements for the course/programme in the Department of Computer Science and Engineering during the academic year 2026–2027.

Student names: RISHI A (2520030268)

SRI CHARAN (2520030093)

SATYA CHARAN (2520030368)

Signature of Guide:

(Dr. S. Vinay Kumar)

Signature of HOD:

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide and the Department of Computer Science and Engineering for their guidance, encouragement, and support throughout the planning of Alumni Connect. We also thank our faculty members, classmates, friends, and families for their suggestions and motivation. This project gives us an opportunity to apply concepts from web development and Data Structures and Algorithms to a practical problem involving alumni engagement, mentoring, and placement support.

Team Members: [Enter names]

ABSTRACT

Educational institutions often maintain alumni information in spreadsheets, emails, and separate records. Because this information is scattered, students may find it difficult to identify alumni who have relevant skills, interests, work experience, or organizational backgrounds. Alumni Connect is a proposed platform that brings student and alumni profiles into one searchable place and helps users discover meaningful connections. The platform is designed to support profile creation, smart search and matching, mentoring requests, job and placement posts, events and groups, messaging, and notifications. A matching engine can compare shared interests, skills, and organizations and rank relevant profiles. The proposed technology stack includes React or React Native for the user interface, Node.js or Django REST API for application services, PostgreSQL for structured records, and a search engine such as Elasticsearch for search. Authentication through institutional identity and role-based access control can help protect user information. The project aims to make alumni engagement more structured, improve access to mentoring and referrals, and provide institutions with a central way to support their alumni community. The report also describes how data structures and algorithms can support profile lookup, filtering, ranking, and network exploration.

TABLE OF CONTENTS

- 1. Introduction
- 1.1 Overview
- 1.2 Problem Statement
- 1.3 Scope of the Project
- 1.4 Objectives
- 2. System Requirements
- 2.1 Functional Requirements
- 2.2 Non-Functional Requirements
- 3. System Design
- 3.1 High-Level Architecture
- 3.2 Description of Layers
- 3.3 Data Flow
- 4. Implementation Plan
- 4.1 Module-Wise Design
- 4.2 Front-End Logic
- 4.3 API and Database Design
- 4.4 Data Structures and Algorithms
- 5. Expected Results and Evaluation
- 6. Challenges and Limitations
- 7. Conclusion and Future Work
- References

Note: This table of contents is a section guide. Update page numbers after final formatting in Microsoft Word.

1. INTRODUCTION

1.1 Overview

Alumni Connect is a proposed web and mobile-friendly platform that brings students and alumni together in a structured digital network. Each user can maintain a profile containing details such as interests, skills, batch, organization, academic background, and professional experience. Students can search for alumni whose experience matches their goals, while alumni can share guidance, mentoring opportunities, job openings, and referrals.

The platform is intended to move alumni engagement beyond occasional reunions or informal messages. By providing searchable profiles and organized interaction features, it can make networking and mentoring easier to discover and manage.

1.2 Problem Statement

Alumni information is often distributed across spreadsheets, emails, social media, and old institutional databases. Students may not know which alumni have relevant skills, work in a particular organization, or are willing to provide career guidance. Without structured search and matching, mentoring and placement connections may happen by chance, and institutions may find it difficult to maintain an active alumni community.

The problem addressed by Alumni Connect is to design a central platform where student and alumni profiles can be searched and matched using shared interests, skills, and organizations, with supporting facilities for mentoring, job opportunities, events, and communication.

1.3 Scope of the Project

The proposed scope includes profile management, searchable student and alumni records, filters and recommendation of relevant connections, mentoring requests, a job and placement board, events and groups, messaging, and notifications. The system may provide separate permissions for students, alumni, and institutional administrators.

The current presentation describes a project proposal and suggested technology stack. Live integrations, authentication, database deployment, message delivery, and production readiness should be treated as planned work unless they have been implemented and tested by the team.

1.4 Objectives

- Create a centralized directory of student and alumni profiles.
- Allow users to search and filter profiles by skills, interests, batch, and organization.
- Rank potential connections using shared profile attributes.
- Provide a structured workflow for sending and responding to mentoring requests.
- Allow alumni to publish job openings, referrals, events, and group activities.
- Support communication and notifications after users discover a connection.
- Apply suitable data structures and algorithms to searching, matching, sorting, and network-related operations.
- Protect user information through authentication, consent, and role-based permissions.

2. SYSTEM REQUIREMENTS

2.1 Functional Requirements

- FR1 – User profiles: users can create and update profiles containing interests, skills, batch, organization, education, and professional details.
- FR2 – Search and filtering: users can search for profiles and narrow results by skills, interests, organization, batch, or role.
- FR3 – Smart matching: the system can calculate a relevance score based on shared profile attributes and display suitable connections.
- FR4 – Mentoring requests: students can send a mentoring request; alumni can accept, decline, and manage requests.
- FR5 – Job and placement board: alumni can publish openings and referral opportunities, and students can view details and follow application instructions.
- FR6 – Events and groups: users can discover reunions, webinars, professional groups, and interest-based communities.
- FR7 – Messaging and notifications: connected users can communicate and receive alerts about requests, messages, and relevant opportunities.
- FR8 – Administration: authorized institutional staff can review reported content and manage records according to defined permissions.
- FR9 – Privacy controls: users can control the profile information shown to other users and request correction of inaccurate information.

2.2 Non-Functional Requirements

Requirement	Description
Usability	A clear interface should be understandable to students and alumni with different levels of technical experience.
Performance	Search and matching should return results quickly for the expected size of the institution's alumni directory.
Security	Authentication, role-based access, input validation, secure password handling, and protected API endpoints should be used.
Privacy	Only necessary profile information should be collected, and visibility should follow user consent and institutional policy.
Reliability	The platform should handle failed requests gracefully and avoid losing profile or mentoring-request updates.
Scalability	The design should allow the number of profiles, job posts, events, and messages to grow.
Maintainability	The application should separate UI components, business logic, API routes, and data-access code.

Accessibility	Forms should have labels, keyboard navigation, readable contrast, and helpful validation messages.

2.3 Suggested Software and Hardware

Suggested software from the project presentation: React or React Native for the frontend; Node.js or Django REST API for backend services; PostgreSQL for structured records; Elasticsearch for search and matching; institutional single sign-on (SSO) and role-based access control for authentication and privacy; and cloud hosting with a CI/CD pipeline. These are proposed choices and can be finalized based on the team's implementation.

Hardware: a laptop or desktop computer, a modern web browser, and an internet connection for development and testing. A server or cloud environment is required if the application is deployed for multiple users.

3. SYSTEM DESIGN

3.1 High-Level Architecture

The proposed architecture separates the user interface, application services, matching logic, database, and authentication layer. Users interact with the frontend. The frontend sends requests to the application server, which validates permissions, reads or updates records, invokes search and matching logic, and returns results.

Layer	Responsibility
Presentation layer	Responsive student and alumni screens, profile forms, search results, job listings, events, mentoring requests, and messages.
Application/API layer	Validates requests and implements profile, mentoring, job, event, and communication workflows.
Matching and search layer	Filters profiles, calculates similarity scores, ranks results, and supports keyword search.
Data layer	Stores profiles, skills, interests, organizations, requests, posts, events, and related records.
Security layer	Handles authentication, roles, authorization, privacy settings, and audit needs.
Deployment layer	Hosts the application and database, manages configuration, backups, and continuous deployment.

3.2 Description of Layers

Presentation Layer

The presentation layer provides separate views for students, alumni, and administrators. Important screens include a dashboard, profile page, search and match results, mentoring requests, job board, events/groups, and messaging.

Application Layer

The application layer manages the main business rules. It checks required fields, validates profile updates, controls request status transitions, and ensures that only authorized users can perform actions such as editing a post or viewing restricted details.

Matching Layer

The matching engine compares a user's search criteria or profile with other profiles. A simple first version can assign weights to shared skills, interests, and organizations, then sort candidates by the resulting score. The scoring method should be documented and tested so users can understand why a profile is recommended.

Data Layer

A relational database such as PostgreSQL can store user records and relationships between profiles, skills, organizations, mentoring requests, jobs, and events. A search index such as Elasticsearch is a suggested option for efficient text search; it is not mandatory for a small prototype.

3.3 Data Flow

1. 1. A user signs in and opens the dashboard.
2. 2. The user searches by a skill, interest, batch, or organization.
3. 3. The application validates the search and retrieves matching profile records.
4. 4. The matching engine scores and ranks relevant profiles.
5. 5. The user views a profile and sends a mentoring request or follows an opportunity.
6. 6. The recipient receives a notification and accepts, declines, or responds.
7. 7. The system stores the action and updates the relevant status.

4. IMPLEMENTATION PLAN

4.1 Module-Wise Design

Module	Main responsibilities
User Profiles	Create, view, edit, and manage student/alumni profile information and privacy settings.
Smart Search & Matching	Search profiles, apply filters, compare shared skills/interests/organizations, and rank results.
Mentoring Requests	Create requests, track pending/accepted/declined states, and support scheduling or progress tracking.
Job & Placement Board	Publish job openings and referrals; browse opportunities and provide application details.
Events & Groups	Create or browse reunions, webinars, and interest-based communities.
Messaging & Notifications	Support conversation flow and alerts for requests, replies, and opportunities.
Admin and Privacy	Manage reports, role permissions, profile verification, and moderation.

4.2 Front-End Logic

React is the suggested frontend framework in the presentation. The interface can be divided into reusable components such as ProfileCard, SearchBar, FilterPanel, MatchList, MentoringRequestForm, JobCard, EventCard, ChatPanel, and NotificationList. Component state can manage search input, selected filters, active tabs, and form status. Client-side validation improves usability, while the server must repeat validation before saving data.

A responsive layout should work on desktop, tablet, and mobile screens. Forms should clearly indicate required fields and show helpful messages when input is missing or invalid.

4.3 API and Database Design

The presentation proposes Node.js or Django REST API for backend services and PostgreSQL for records. Example API groups may include /profiles, /search, /matches, /mentoring-requests, /jobs, /events, /groups, /messages, and /notifications. These are design examples, not claims that the endpoints already exist.

Entity	Example fields
User/Profile	user_id, role, name, batch, skills, interests, organizations, visibility
MentoringRequest	request_id, student_id, alumni_id, message, status, created_at
JobPost	job_id, alumni_id, title, organization, description, application_details

Event	event_id, title, event_type, date_time, location_or_link, organizer_id
Group	group_id, name, description, created_by
Message	message_id, conversation_id, sender_id, message_text, created_at
Notification	notification_id, user_id, type, reference_id, read_status, created_at

Database constraints should prevent invalid references and inconsistent request states. Passwords should never be stored as plain text. Access to profile details, messages, and contact information should follow role and privacy settings.

4.4 Data Structures and Algorithms

The project can demonstrate Data Structures and Algorithms concepts in practical operations. The following are suitable design choices; the final report should identify which algorithms were actually implemented in the team's code.

Concept	Possible use in Alumni Connect
Hash map	Retrieve a profile or user record quickly by unique ID.
Arrays/lists	Store and display search results, skills, job posts, and event lists.
Sorting	Rank profile matches by similarity score, experience, or user-selected criteria.
Searching/indexing	Find profiles using names, skills, interests, and organizations.
Set operations	Compare skill or interest sets to identify shared attributes.
Graph representation	Represent mentoring or professional connections as users and relationships.
Queue	Process notifications or pending requests in order where appropriate.
Fuzzy string matching	Handle small spelling variations in names, skills, or organizations.

A simple matching score can be defined as a weighted combination of shared attributes. For example: Match Score = w1 × skill similarity + w2 × interest similarity + w3 × organization similarity, where the weights w1, w2, and w3 sum to 1. The score can be converted to a percentage and used to sort candidates. The team should select weights based on project requirements and explain that the score indicates relevance, not a guarantee of compatibility.

5. EXPECTED RESULTS AND EVALUATION

Based on the project proposal, Alumni Connect is expected to make alumni information easier to discover and to create a more structured route from profile discovery to mentoring and placement opportunities. The intended outcomes are summarized below; these are expected outcomes, not measured results.

Area	Expected outcome	How to evaluate
Profile discovery	Students can find alumni using searchable profile information.	Test searches with known names, skills, and organizations.
Matching	Relevant profiles appear near the top of the results.	Use sample profiles and manually verify ranking for selected cases.
Mentoring	Requests have clear pending, accepted, or declined states.	Test request creation and each status transition.
Placements	Job posts can be published and browsed.	Test posting, listing, filtering, and opening job details.
Events and groups	Users can discover relevant activities and communities.	Test creation, display, and filtering of events/groups.
Security and privacy	Users see only the information permitted by their role and settings.	Test unauthorized access and profile visibility rules.
Usability	Users can complete common tasks with limited confusion.	Conduct a short task-based test with classmates.

5.1 Sample Test Cases

Test	Input/action	Expected result
Search by skill	Search for “Java”	Profiles with Java in searchable skill fields are returned.
Filter by organization	Choose a particular organization	Only profiles matching the selected organization remain.
Send mentoring request	Student submits a valid request	Request is saved with pending status and visible to the recipient.
Accept request	Alumnus accepts a pending request	Status changes to accepted and the requester is notified.
Unauthorized access	A user attempts to edit another user's profile	The server rejects the action.
Empty required field	Submit a profile form without a required field	Validation explains what must be completed.

No numerical performance or accuracy results are claimed in this document because the supplied presentation describes a project proposal and does not provide measured test results. Replace this section with screenshots, actual test outcomes, and timings after implementation.

6. CHALLENGES AND LIMITATIONS

6.1 Anticipated Challenges

- Keeping alumni records accurate and up to date when information comes from different sources.
- Choosing a matching method that is useful, transparent, and not biased toward a single profile attribute.
- Protecting contact details, messages, and other personal information.
- Preventing spam, fake profiles, and inappropriate job or event posts.
- Handling duplicate profiles and inconsistent names for skills or organizations.
- Maintaining performance as the number of users, messages, and posts grows.
- Ensuring that the interface remains simple for users with different levels of technical experience.

6.2 Proposed Solutions

- Use profile verification, update reminders, and correction/reporting options to improve data quality.
- Document the matching score and let users filter results instead of relying only on recommendations.
- Use authentication, role-based access, secure password hashing, validation, and privacy controls.
- Provide moderation tools, reporting options, and clear rules for job and event submissions.
- Normalize common skills and organizations and support duplicate detection.
- Use database indexes, pagination, and monitoring as the dataset grows.
- Test common tasks with students and alumni and revise the interface based on feedback.

6.3 Limitations

The project presentation outlines proposed features and a suggested technology stack, but it does not establish that every module has already been built. The quality of matching depends on profile completeness and the scoring rules. The usefulness of the platform also depends on alumni participation, accurate data, and institutional support. Messaging, notifications, authentication, and deployment require additional implementation and security testing before real-world use.

7. CONCLUSION AND FUTURE WORK

7.1 Conclusion

Alumni Connect proposes a centralized platform where students and alumni can discover one another based on skills, interests, batches, and organizations. By combining searchable profiles with mentoring requests, job and placement posts, events, groups, messaging, and notifications, the project aims to turn informal alumni contact into a more organized network. The design also provides opportunities to apply data structures and algorithms to search, matching, sorting, and relationship management. The next step is to implement the core modules, validate them with sample users, and measure the quality and usability of the results.

7.2 Future Work

- Complete and deploy the profile, search, and matching modules.
- Add institutional single sign-on and profile verification.
- Improve recommendations using feedback and more detailed matching criteria.
- Provide calendar integration and scheduling for mentoring sessions.
- Add job application tracking and alumni referral workflows.
- Introduce analytics for engagement and placement outcomes while protecting privacy.
- Improve mobile usability and accessibility.
- Add automated tests, backups, monitoring, and a CI/CD pipeline.

7.3 Skills to Be Developed

The project provides an opportunity to practise frontend development, API design, database modelling, authentication and authorization, search and ranking, data structures and algorithms, testing, version control, teamwork, and technical documentation. The team should update this subsection to describe the skills actually learned during implementation.

REFERENCES

[1] React Documentation. https://react.dev/

[2] React Native Documentation. https://reactnative.dev/docs/getting-started

[3] Node.js Documentation. https://nodejs.org/docs/

[4] Django REST framework Documentation. https://www.django-rest-framework.org/

[5] PostgreSQL Documentation. https://www.postgresql.org/docs/

[6] Elasticsearch Documentation. https://www.elastic.co/guide/

[7] OWASP Foundation, OWASP Top 10. https://owasp.org/www-project-top-ten/

[8] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed., MIT Press, 2022.

Note: The technology choices above follow the Alumni Connect presentation. Cite any additional libraries, APIs, datasets, or external material actually used during implementation.